Python Strings

Dr. Ravi B, Dep. of CSE

Python Strings

- **String literal**
- **Multiline strings**
- **Accessing Elements**
- **Membership operator with strings**
- **Using + and * with strings**
- **Using Format()**
- **Escape Characters**
- **String methos**

Python Strings – String literal

- String literals in python are surrounded by either single quotation marks, or double quotation marks.

  'hello' is the same as "hello"

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

```
a = "Hello"
print(a)
```

Python Strings - multiline strings

- You can assign a multiline string to a variable by using three quotes (double or single):

Example-1

```
a = """Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."""

print(a)
```

Note: in the result, the line breaks are inserted at the same position as in the code.

Python Strings - multiline strings

- Following example shows multiline string surrounded by triple single quotes

Example-2

```
a = '''Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua'''

print(a)
```

Python Strings - Accessing Elements

- Strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1
- Square brackets can be used to access elements of the string.

Example

```
a = "Hello, World!"
print(a[1])
```

Output:

```
e
```

Python Strings - Accessing Elements

- You can return a range of characters by using **string slicing**
- Specify the start index and the end index, separated by a colon, to return a part of the string.

Example :Get the characters from position 2 to position 5 (not included):

```
b = "Hello, World!"
print(b[2:5])
```

Output:

```
llo
```

Python Strings - Accessing Elements

- **The term 'string slice' refers to a part of the string, where strings are sliced using a range of indices. For a string say name[n : m] where n and m are integers and legal indices. Python will return a slice of the string by returning the characters falling between indices n and m starting at n,n+1,n+2... till m-1.**

Let"s understand Slicing in strings with the help of few examples.

String word	a	m	a	z	i	n	g
Positive Index	0	1	2	3	4	5	6
Negative Index	-7	-6	-5	-4	-3	-2	-1

Python Strings - Accessing Elements

- **Slice From the Start**

Get the characters from the start to position 5 (not included):

```
b = "Hello, World!"
print(b[:5])
```

Output:
Hello

Python Strings - Accessing Elements

- **Slice To the End**

Get the characters from position 2, and all the way to the end:

```
b = "Hello, World!“
print(b[2:])
```

Output:

llo, World!

Python Strings - Accessing Elements

- **Negative Indexing :** Use negative indexes to start the slice from the end of the string: **(Count from -1 from the end for negative index)**

Example : Get the characters from position -5 to position -2 (not included)

```
b = "Hello, World!"
print(b[-5:-2])
```

Output:

```
orl
```

Python Strings - Accessing Elements

- **More Example for string slicing :**

```
x = "Hello, Python"
print(x[-5:])

print(x[:-3])

print(x[7:-1])

print(x[:])
```

Output:
ython

Output:
Hello, Pyt

Output:
Pytho

Output:
Hello, Python

Python Strings - Accessing Elements

- You can also include a third number in slicing to indicate skipping step. In the following example, we start from index 1 and end with index 6 index(stops at 6-1=5), and the skipping step is 2. It is a good example of Python slicing string by character.

```
x = "Hello, Python"
print(x[1:6:2])
```

Output:

```
el,
```

Python Strings - Accessing Elements

- Also when skipping step is negative, string slicing is reversed. In this case first number indicates start index in the reverse order, second number is end index (not included) in the reverse order, and the last number is skipping step in the reverse order.
- Examples :

```
x = "Hello, Python"
print(x[4:1:-1])
```

Output:

```
oll
```

Python Strings - Accessing Elements

- Examples :

```
x = "Hello, Python"
print(x[5:0:-2])

print(x[8: : -2])

print(x[: 6: -2])

print(x[ : :-1])
```

Output:
,le

Output:
y olH

Output:
nhy

Output:
nohtyP ,olleH

Python Strings - Accessing Elements

- Examples :

```
x = "Hello, Python"
print(x[-2:7:-1])

print(x[4:-12:-1])

print(x[-2:-6:-1])
```

Output:
ohty

Output:
oll

Output:
ohty

Python Strings – Getting String Length

- To get the length of a string, use the len() function

Example :

```
a = "Hello, World!"
print(len(a))
```

Output:
13

Python string - membership operator

- To check if a certain phrase or character is present in a string, we can use the keywords **in or not in.**

Example : Check if the phrase "ain" is present in the following text:

```
txt = "The rain in Spain stays mainly in the plain"
x = "ain" in txt
print(x)
x = "ain" not in txt
print(x)
```

Output:

```
True
False
```

Python **Strings – using +**

- To concatenate, or combine, two strings you can use the + operator.

- Example :

```
a = "Hello"
b = "World"
c = a + b
print(c)
```

Output:
HelloWorld

Python **Strings - using +**

Example :

To add a space between them, add a " ":

```
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```

Output:

Hello World

Python **Strings - using *

- (Replication) operator: The * operator replicates the string the number of times the value given on the other side.

- Consider the following example >>>3*"go " "gogogo"

Python **Strings - format()**

- The format() method takes unlimited number of arguments, and are placed into the respective placeholders:

Example :

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want {} pieces of item {} for {} dollars."
print(myorder.format(quantity, itemno, price))
```

Output:

```
I want 3 pieces of item 567 for 49.95 dollars.
```

Python **Strings - format()**

- You can use index numbers inside {} to make sure that arguments are placed in the correct placeholders:

Example :

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want to pay {2} dollars for {0} pieces of
item {1}."
print(myorder.format(quantity, itemno, price))
```

Output:

I want to pay 49.95 dollars for 3 pieces of item 567

Python escape character

- To insert characters that are illegal in a string, use an escape character.
- An escape character is a backslash \ followed by the character you want to insert.

Example :

```
txt = "We are the so-called \"Vikings\" from the north."
Print(txt)
```

Output:
We are the so-called "Vikings" from the north.

Python escape character

- Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

Python **String Formatting Operator**

- One of Python's coolest features is the string format operator %.
- This operator is unique to strings and makes up for the pack of having functions from C's printf() family. **Following is a simple example –**

```
print "My name is %s and weight is %d kg!" % ('Zara', 21)
```

Outpout : My name is Zara and weight is 21 kg!

Python **String Formatting Operator**

- Here is the list of complete set of symbols which can be used along with % –

Format Symbol	Conversion
%c	character
%s	string conversion via str() prior to formatting
%i	signed decimal integer
%d	signed decimal integer
%u	unsigned decimal integer
%o	octal integer
%x	hexadecimal integer (lowercase letters)
%X	hexadecimal integer (UPPERcase letters)

Python **String Formatting Operator**

- Here is the list of complete set of symbols which can be used along with % –

%e	exponential notation (with lowercase 'e')
%E	exponential notation (with UPPERcase 'E')
%f	floating point real number
%g	the shorter of %f and %e
%G	the shorter of %f and %E

Python String - methods

- The strip() method removes any whitespace from the beginning or the end:

Example :

```
a = "    Hello, World!   "
print(a.strip())  # returns "Hello, World!"
```

Python String - methods

- The lower() and upper() method returns the string in lower and upper case respectively:

Example :

```
a = "Hello, World!"
print(a.lower())
print(a.upper())
```

Output:

```
hello, world!
HELLO, WORLD!
```

Python String - methods

- The replace() method replaces a string with another string:

Example :

```
a = "Hello, World!"
print(a.replace("H", "J"))
```

Output:

Jello, World!

Python String - methods

- The split() method splits the string into substrings if it finds instances of the separator:

Example :

```
a = "Hello, World!"
print(a.split(",")) # returns ['Hello', ' World!']
```

Output:

```
['Hello', ' World!']
```

Python String - All methods

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value

Python String - All methods

expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters

Python String - All methods

isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title

Python String - All methods

isupper()	Returns True if all characters in the string are upper case
join()	Converts the elements of an iterable into a string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts

Python String - All methods

rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value

Python String - All methods

strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning